```
        Countries = Inputs
            .Where(x => !string.IsNullOrWhiteSpace(x.CountryCode))
            .Select(x => new Country {
            CountryCode = x.CountryCode,
            CountryName = x.CountryName
        }).ToList();
    }
}
```

Next, you need to change the data entry form to accept multiple countries instead of just one at a time. You will render five rows in a table, each containing inputs for the country name and its ISO-3166-1 code. You need to ensure that the format of the `name` attribute's value on each of the input controls adopts the following pattern:

```
<property>[index].<property>
```

This is the same pattern as for a single complex object. The only change is the incorporation of the indexer. So the rendered HTML for the inputs should end up looking like the following listing (removing the extraneous markup).

Listing 5.17 Rendered markup for multiple inputs using a sequential index

```
<input name="Inputs[0].CountryName" /><input name="Inputs[0].CountryCode" />
<input name="Inputs[1].CountryName" /><input name="Inputs[1].CountryCode" />
<input name="Inputs[2].CountryName" /><input name="Inputs[2].CountryCode" />
<input name="Inputs[3].CountryName" /><input name="Inputs[3].CountryCode" />
<input name="Inputs[4].CountryName" /><input name="Inputs[4].CountryCode" />
```

The following listing shows the code that generates those inputs, using a simple `for` loop that starts with zero.

Listing 5.18 Using a sequential index to enable multiple data entry

```
@page
@model CityBreaks.Pages.CountryManager.CreateModel
```